

# PULSE Framework – TikTok Batch 1

## 20 Videos optimiert für Performance

Manuel Fuß | Head of AI, RSLT.DIGITAL | Januar 2026

---

### Performance-Regeln (aus YouTube Analytics)

- ☒ Problem-Framing schlägt Lösungs-Framing
- ☒ Konkrete Zahlen im Titel (3, 4, 5...)
- ☒ Trigger-Wörter: Kontrolle, Fehler, Probleme, Loops
- ☒ Sweet Spot: 30–45 Sekunden (TikTok)

### Standard-Intro

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

---

## Block 1: Kontrollverlust-Momente

### Videos 1–5: Wann es schiefgeht

---

#### Video 1: "Der Moment, in dem die KI dein Projekt zerstört"

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "Das passiert schneller als du denkst"

*"Ein falscher Klick auf 'Apply All' – und 47 Dateien sind geändert. Rollback unmöglich."*

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

*"Der Agent macht einen Vorschlag. Du klickst Apply. Dann noch einen. Und noch einen. Irgendwann merkst du: Das Projekt kompiliert nicht mehr. Du schaust in Git – 47 Dateien geändert, kein sauberer Checkpoint. Du weißt nicht, welche Änderung das Problem ist. Und jetzt hast du die Wahl: Alles manuell durchgehen – oder alles wegwerfen."*

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

*"Die Lösung: Commit alle fünf Minuten. Kein Commit, kein Apply. Folg mir für mehr."*

---

#### Video 2: "3 Zeichen, dass du die Kontrolle verloren hast"

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "Wer steuert hier eigentlich?"

*"Woran merkst du, dass nicht mehr du das Projekt steuerst – sondern die KI?"*

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

*"Zeichen eins: Du scrollst durch den Code und denkst – was macht das eigentlich? Du verstehst deinen eigenen Code nicht mehr. Zeichen zwei: Du akzeptierst jeden Vorschlag, weil neu anfangen zu aufwändig ist. Zeichen drei: Der Agent installiert Dependencies, die du nicht kennst – und du fragst nicht nach warum."*

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

*"Wenn du bei einem dieser Zeichen nickst – Zeit für einen Reset. Folg mir für mehr."*

---

## **Video 3: "Warum dein KI-Agent gerade im Kreis läuft"**

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "Täglich grüßt das Murmeltier"

*"Die KI sagt: Gefixt. Du testest: Geht nicht. Die KI sagt: Jetzt gefixt. Du testest: Geht immer noch nicht."*

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

*"Das ist ein Fix-Loop – der häufigste Fehler in der KI-Entwicklung. Die KI probiert dieselbe Strategie immer wieder, weil sie kein echtes Verständnis hat. Sie ändert eine Zeile, hofft dass es klappt, und wenn nicht – ändert sie eine andere Zeile. Nach dem dritten Versuch: Stop. Nicht weitermachen. Das Problem braucht einen Menschen."*

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

*"Drei Versuche, dann eskalieren. Keine Ausnahmen. Folg mir für mehr."*

---

## **Video 4: "Der gefährlichste Moment in der KI-Entwicklung"**

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "Der Moment, den alle unterschätzen"

*"Es gibt einen Moment, der mehr Projekte zerstört als alle Bugs zusammen."*

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

*"Der gefährlichste Moment? Wenn es zu gut läuft. Die KI baut Feature nach Feature, du lehnst dich zurück, alles funktioniert. Und genau dann hörst du auf, jeden Edit zu prüfen. Du wirst bequem. Und zwei Stunden später hast du tausend Zeilen Code, die du nicht verstehst – und keinen Checkpoint zum Zurückspringen."*

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

*"Gerade wenn es läuft – bleib wachsam. Folg mir für mehr."*

---

## Video 5: "Was passiert nach 30 Minuten ohne Checkpoint"

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "Was nach 30 Minuten passiert"

*"Was passiert eigentlich, wenn du den Agent länger als 30 Minuten alleine lässt?"*

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

*"Der Agent verliert den Kontext. Er macht Änderungen, die nicht mehr zum ursprünglichen Ziel passen. Er baut Features, die du nie wolltest. Er überschreibt funktionierende Sachen, weil er denkt, er macht sie besser. Und am Ende hast du mehr Arbeit als vorher – weil du alles rückgängig machen musst."*

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

*"30 Minuten Maximum. Keine Ausnahmen. Folg mir für mehr."*

---

## Block 2: Die 4 Loop-Typen

Videos 6–10: Loops erkennen und stoppen

---

### Video 6: "Loop-Typ 1: Die KI behauptet es sei gefixt"

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "'Problem gelöst' – oder doch nicht?"

*"Commit-Message: 'Fix: Problem gelöst'. Du testest: Problem ist noch da."*

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

*"Das ist Loop-Typ eins – der häufigste. Die KI committed 'fix: Problem gelöst', du testest, Problem ist noch da. Sie committed wieder 'fix: Jetzt wirklich gefixt'. Immer noch da. Nach dem zweiten Mal: Stop. Die KI versteht das Problem nicht. Reject die Commits, erkläre das Problem neu – oder eskaliere zu einem stärkeren Modell."*

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

*"Zwei Fix-Commits zum gleichen Problem? Sofort stoppen. Folg mir für mehr."*

---

## **Video 7: "Loop-Typ 2: Dieselbe Datei wird 5x geändert"**

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "File Churn"

*"Die KI ändert config.ts. Dann nochmal. Und nochmal. Fünfmal dieselbe Datei."*

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

*"Das ist Loop-Typ zwei – File Churn. Die KI findet keine stabile Lösung. Sie ändert, testet, ändert zurück, testet wieder. Jede Änderung macht es schlimmer. Du erkennst es daran: Dieselbe Datei taucht in drei oder mehr aufeinanderfolgenden Commits auf. Das ist das Signal zum Stoppen."*

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

*"Drei Commits, dieselbe Datei? Loop erkannt. Stoppen. Folg mir für mehr."*

---

## **Video 8: "Loop-Typ 3: Die KI macht ihre eigenen Fixes rückgängig"**

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "Hin und zurück"

*"Commit 1: Feature hinzugefügt. Commit 2: Feature entfernt. Commit 3: Feature wieder hinzugefügt."*

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

"Das ist Loop-Typ drei – das Pendel. Die KI ist unsicher. Sie fügt Code hinzu, dann entfernt sie ihn wieder, dann fügt sie ihn anders hinzu. Sie kann sich nicht entscheiden. Und du zahlst den Preis: Jeder Commit ist ein Risiko, jeder Rollback ein Abenteuer. Wenn du das Muster siehst: Sofort eingreifen."

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

"Pendel-Pattern erkannt? Die KI braucht klarere Anweisungen. Folg mir für mehr."

---

## Video 9: "Loop-Typ 4: 10 Dateien geändert, 5 neue Fehler"

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "Scope Explosion"

"Du wolltest einen Button fixen. Jetzt sind zehn Dateien geändert – und fünf neue Fehler."

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

"Das ist Loop-Typ vier – Scope Explosion. Die KI macht zu viel auf einmal. Du fragst nach einem Fix, sie refactored das halbe Projekt. Und jetzt weißt du nicht: War es Datei zwei? Datei fünf? Die Kombination? Du kannst es nicht mehr eingrenzen. Der Schaden ist größer als das ursprüngliche Problem."

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

"Kleine Schritte, eine Datei nach der anderen. Folg mir für mehr."

---

## Video 10: "So erkennst du in 5 Sekunden ob du in einem Loop steckst"

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "Der 5-Sekunden-Check"

"Fünf Sekunden, ein Blick – und du weißt, ob dein Agent im Loop ist."

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

*"Git log anschauen. Letzte fünf Commits. Frag dich: Taucht dieselbe Datei mehrfach auf? Steht zweimal 'fix' in der Message? Wurde Code hinzugefügt und dann wieder entfernt? Sind mehr Dateien betroffen als nötig? Wenn du einmal Ja sagst: Warnung. Zweimal Ja: Stop. Das ist ein Loop."*

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

*"Fünf Sekunden Check, jeden Tag. Folg mir für mehr."*

---

## Block 3: Die größten Fehler

**Videos 11–15: Was 90% falsch machen**

---

### Video 11: "Der Fehler, den 90% beim ersten Prompt machen"

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "90% machen das falsch"

*"Dein erster Prompt entscheidet über Erfolg oder Frust – und die meisten versauen ihn."*

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

*"Der Fehler: Zu viel auf einmal. 'Bau mir eine Login-Page mit Authentifizierung, Passwort-Reset, OAuth und E-Mail-Verifizierung.' Das ist kein Prompt – das ist eine Feature-Liste. Die KI rät, wo sie anfangen soll. Du bekommst ein Chaos. Stattdessen: Ein Feature, ein Prompt. Login first. Auth second. Immer."*

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

*"Ein Feature, ein Prompt. Keine Ausnahmen. Folg mir für mehr."*

---

### Video 12: "3 Wörter, die jeden KI-Prompt ruinieren"

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "Diese 3 Wörter vermeiden"

*"Drei Wörter, die deinen Prompt garantiert ruinieren."*

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

#### [0:08–0:28] Content

##### CAM 2 (Seitlich)

*"'Mach es besser.' Was ist besser? Schneller? Schöner? Lesbarer? Die KI rät – und rät falsch. 'Irgendwie so.' Irgendwie wie? Die KI interpretiert – und interpretiert falsch. 'Fix alles.' Alles? Jeden Lint-Error? Jeden TODO-Kommentar? Die KI übertreibt – und zerstört funktionierenden Code."*

#### [0:29–0:35] Takeaway

##### CAM 1 (Frontal)

*"Vage Prompts, vage Ergebnisse. Sei spezifisch. Folg mir für mehr."*

---

## Video 13: "Warum dein Code funktioniert – aber trotzdem falsch ist"

#### [0:00–0:03] Hook

##### CAM 2 (Seitlich)

Text-Overlay: "Funktioniert ≠ Richtig"

*"Der Code läuft. Die Tests sind grün. Und trotzdem ist er falsch."*

#### [0:04–0:07] Intro

##### CAM 1 (Frontal)

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

#### [0:08–0:28] Content

##### CAM 2 (Seitlich)

*"Die KI nimmt Abkürzungen. Sie hardcodes Werte statt Variablen. Sie kopiert Code statt ihn zu abstrahieren. Sie ignoriert Edge Cases. Jetzt funktioniert es – aber in drei Monaten bricht alles. Das nennt sich Tech Debt. Und KI baut Tech Debt schneller auf als jeder Junior-Entwickler."*

#### [0:29–0:35] Takeaway

##### CAM 1 (Frontal)

*"Funktioniert ist nicht genug. Versteh den Code. Folg mir für mehr."*

---

## Video 14: "Der teuerste Fehler: Zu lange warten mit dem Stoppen"

#### [0:00–0:03] Hook

##### CAM 2 (Seitlich)

Text-Overlay: "Der teuerste Fehler"

*"Du siehst, dass was schief läuft – aber du denkst: Vielleicht fixt er's noch."*

#### [0:04–0:07] Intro

##### CAM 1 (Frontal)

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

#### [0:08–0:28] Content

##### CAM 2 (Seitlich)

*"Der teuerste Fehler ist nicht der Loop. Nicht der Bug. Nicht der falsche Prompt. Der teuerste Fehler ist Warten. Jede Minute, die du den Agent weitermachen lässt, obwohl du weißt dass was nicht stimmt – ist*

*eine Minute, die du später reparieren musst. Und Reparieren dauert dreimal so lang wie Stoppen."*

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

*"Bauchgefühl sagt Stop? Dann Stop. Sofort. Folg mir für mehr."*

---

## Video 15: "Warum erfahrene Entwickler bei KI-Tools scheitern"

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "10 Jahre Erfahrung – und trotzdem frustriert"

*"Warum geben gerade erfahrene Entwickler bei KI-Tools so schnell auf?"*

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

*"Weil sie erwarten, dass es wie ein Compiler funktioniert. Befehl rein, Ergebnis raus. Deterministisch. Aber KI ist kein Befehlsempfänger – KI ist ein Denkpartner. Du sagst was, sie macht was, du korrigierst, sie passt an. Das ist ein Dialog, kein Monolog. Wer das versteht, wird schneller als je zuvor."*

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

*"Mindset wechseln: Dialog statt Befehl. Folg mir für mehr."*

---

## Block 4: Red Flags & Safeguards

**Videos 16–20: Warnsignale und Schutzmaßnahmen**

---

### Video 16: "7 Red Flags im Code – sofort stoppen"

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "Sofort handeln"

*"Sieben Warnsignale im Code, bei denen du sofort eingreifen musst."*

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

*"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."*

**[0:08–0:32] Content**

**CAM 2 (Seitlich)**

*"Eins: Code, den du nicht verstehst. Zwei: Hunderte Zeilen in einem Commit. Drei: Dependencies, die du nicht kennst. Vier: Gelöschte Dateien ohne Rückfrage. Fünf: Production-URLs hardcoded im Code."*



Sechs: `console.log` überall. Sieben: API-Keys im Repository. Jedes davon ist ein Grund, sofort zu stoppen."

#### **[0:33–0:38] Takeaway**

##### **CAM 1 (Frontal)**

"Red Flag gesehen? Stoppen, nicht ignorieren. Folg mir für mehr."

---

## **Video 17: "Die 5 Dinge, die KI niemals alleine tun darf"**

#### **[0:00–0:03] Hook**

##### **CAM 2 (Seitlich)**

Text-Overlay: "Non-negotiable"

"Es gibt fünf Dinge, die dein KI-Agent niemals eigenständig tun sollte."

#### **[0:04–0:07] Intro**

##### **CAM 1 (Frontal)**

"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."

#### **[0:08–0:30] Content**

##### **CAM 2 (Seitlich)**

"Eins: Dateien löschen ohne Nachfrage. Zwei: Git Push ohne Erlaubnis. Drei: Direkt auf Production deployen. Vier: Breaking Changes ohne Warnung. Fünf: Secrets in den Code schreiben. Das sind die fünf Critical Safeguards – und sie sind nicht verhandelbar. Egal wie gut der Agent ist."

#### **[0:31–0:37] Takeaway**

##### **CAM 1 (Frontal)**

"Diese fünf Regeln gehören in jede Cursorrules. Folg mir für mehr."

---

## **Video 18: "Die KI hat gerade einen API-Key gepusht"**

#### **[0:00–0:03] Hook**

##### **CAM 2 (Seitlich)**

Text-Overlay: "Das passiert öfter als du denkst"

"Du schaust in den Commit – und da steht dein API-Key. Im Klartext. Im öffentlichen Repo."

#### **[0:04–0:07] Intro**

##### **CAM 1 (Frontal)**

"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."

#### **[0:08–0:28] Content**

##### **CAM 2 (Seitlich)**

"Die KI weiß nicht, was ein Secret ist. Für sie ist 'sk-abc123' einfach ein String. Sie kopiert ihn in den Code, weil er funktioniert. Und wenn du nicht aufpasst, landet er in Git. Für immer. Selbst wenn du ihn löschst – er ist in der Git-History. Bots scannen GitHub nach genau solchen Leaks."

#### **[0:29–0:35] Takeaway**

##### **CAM 1 (Frontal)**

"Secret Detection einrichten. Heute noch. Folg mir für mehr."

---

## Video 19: "Wann du alles stoppen und neu anfangen solltest"

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "Der Punkt ohne Wiederkehr"

"Manchmal ist Weitermachen der größte Fehler. Wann solltest du alles stoppen?"

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

"Wenn du durch die Diffs scrollst und nicht mehr weißt, was zusammenhängt. Wenn neue Errors schneller auftauchen als alte verschwinden. Wenn du mehr Zeit mit Debugging verbringst als mit Building. Dann ist Rollback die bessere Wahl als Weiterwursteln. Zurück zum letzten sauberen Checkpoint."

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

"Manchmal ist Neustart schneller als Reparatur. Folg mir für mehr."

---

## Video 20: "Eine Regel, die alles verändert"

**[0:00–0:03] Hook**

**CAM 2 (Seitlich)**

Text-Overlay: "Die eine Regel"

"Eine Regel. Mehr brauchst du nicht, um KI-Entwicklung zu meistern."

**[0:04–0:07] Intro**

**CAM 1 (Frontal)**

"Hi, ich bin Manuel – ich unterrichte KI-gestützte Entwicklung nach dem PULSE Framework."

**[0:08–0:28] Content**

**CAM 2 (Seitlich)**

"Die Regel: Versteh jeden Commit, bevor du weitermachst. Nicht überfliegen. Verstehen. Was hat sich geändert? Warum? Macht das Sinn? Wenn du eine dieser Fragen nicht beantworten kannst – nicht weitermachen. Das ist der Unterschied zwischen Entwicklern, die mit KI schneller werden, und denen, die im Chaos versinken."

**[0:29–0:35] Takeaway**

**CAM 1 (Frontal)**

"Verstehen, dann weitermachen. Das ist alles. Folg mir für mehr."

---

# Übersicht: Videos 1–20

## Block 1: Kontrollverlust-Momente (1–5)

1. Der Moment, in dem die KI dein Projekt zerstört
2. 3 Zeichen, dass du die Kontrolle verloren hast
3. Warum dein KI-Agent gerade im Kreis läuft
4. Der gefährlichste Moment in der KI-Entwicklung
5. Was passiert nach 30 Minuten ohne Checkpoint

## Block 2: Die 4 Loop-Typen (6–10)

6. Loop-Typ 1: Die KI behauptet es sei gefixt
7. Loop-Typ 2: Dieselbe Datei wird 5x geändert
8. Loop-Typ 3: Die KI macht ihre eigenen Fixes rückgängig
9. Loop-Typ 4: 10 Dateien geändert, 5 neue Fehler
10. So erkennst du in 5 Sekunden ob du in einem Loop steckst

## Block 3: Die größten Fehler (11–15)

11. Der Fehler, den 90% beim ersten Prompt machen
12. 3 Wörter, die jeden KI-Prompt ruinieren
13. Warum dein Code funktioniert – aber trotzdem falsch ist
14. Der teuerste Fehler: Zu lange warten mit dem Stoppen
15. Warum erfahrene Entwickler bei KI-Tools scheitern

## Block 4: Red Flags & Safeguards (16–20)

16. 7 Red Flags im Code – sofort stoppen
17. Die 5 Dinge, die KI niemals alleine tun darf
18. Die KI hat gerade einen API-Key gepusht
19. Wann du alles stoppen und neu anfangen solltest
20. Eine Regel, die alles verändert